
SERVICIO NACIONAL DE APRENDIZAJE — SENA

Centro de Electricidad, Electrónica y Telecomunicaciones — CEET

Guía de Actividad Práctica

CONSTRUCCIÓN DE UNA API REST

PARTE II — Continuidad de la actividad

CRUD de objetos y colecciones · Programación asíncrona · Validaciones · Pruebas · Seguridad con JWT (Roles y Políticas)

Transferencia de conceptos de ASP.NET Core Web API (C#) a un ecosistema tecnológico alternativo, sobre la solución ya construida con Onion Architecture, Repository Pattern y Unit of Work.

Material para aprendices

Modalidad: trabajo en grupo de 2 aprendices · Entrega documentada

Importante: cada grupo continúa con el lenguaje y framework asignado en la Parte I.

Programa de formación: _____ Ficha: _____

Tabla de contenido

Nota sobre la tabla de contenido

Si la tabla aparece vacía, haga clic derecho sobre ella y seleccione «Actualizar campos» (o presione F9) en Microsoft Word. Esto generará automáticamente los números de página.

Presentación — De la Parte I a la Parte II.....	5
1. Objetivo general.....	5
2. Objetivos específicos.....	6
3. Recordatorio — Tecnologías asignadas.....	6
4. Prompts obligatorios para usar IA (Fase II).....	7
Prompt 7 — CRUD de colecciones.....	7
Prompt 8 — Programación asíncrona.....	7
Prompt 9 — Validaciones.....	7
Prompt 10 — Pruebas.....	7
Prompt 11 — JWT por roles.....	8
Prompt 12 — JWT por políticas	8
5. Módulo 1 — CRUD completo: objetos y colecciones	8
5.1 Objeto vs. colección	8
5.2 Operaciones sobre colecciones.....	8
5.3 Paginación, filtrado y ordenamiento	9
5.4 Endpoints adicionales requeridos	9
5.5 Códigos HTTP para colecciones	10
5.6 Pasos del módulo	10
6. Módulo 2 — Programación asíncrona	11
6.1 ¿Por qué importa?	11
6.2 Investigación por tecnología	11
6.3 Refactorización: de sincrónico a asíncrono.....	12
6.4 Async y Unit of Work.....	12
6.5 Pasos del módulo	12
7. Módulo 3 — Validaciones	13

7.1 ¿Por qué validar en la capa de aplicación?	13
7.2 Librería de validación por tecnología.....	13
7.3 Validaciones mínimas requeridas	14
7.4 Manejo de errores de validación	14
7.5 Pasos del módulo	14
8. Módulo 4 — Pruebas	15
8.1 Pirámide de pruebas	15
8.2 Framework de pruebas por tecnología	15
8.3 Qué se debe probar	15
8.4 Prueba del caso transaccional (rollback).....	15
8.5 Pasos del módulo	16
9. Módulo 5 — Seguridad: JWT orientado a Roles	16
9.1 Autenticación vs. autorización	16
9.2 ¿Qué es un JWT?.....	16
9.3 Nuevas entidades: Usuario y Rol	17
9.4 Flujo de autenticación	17
9.5 Endpoints de autenticación	17
9.6 Roles por tecnología.....	18
9.7 Ubicación en Onion Architecture	18
9.8 Pasos del módulo	18
10. Módulo 6 — Seguridad: JWT orientado a Políticas (Policies)	19
10.1 Roles vs. políticas.....	19
10.2 Autorización basada en claims y reglas.....	19
10.3 Políticas a implementar.....	19
10.4 Políticas por tecnología.....	19
10.5 Comparación con ASP.NET Core	20
10.6 Pasos del módulo	20
11. Producto final esperado (Parte II).....	21
12. Actualización del README.....	21
13. Criterios de evaluación (Parte II)	21
14. Comparación ampliada con ASP.NET Core	22
15. Preguntas para sustentación (Parte II).....	22
16. Reglas técnicas obligatorias (Parte II)	23

Resultado esperado de aprendizaje	23
---	----

Presentación — De la Parte I a la Parte II

En la **Parte I** cada grupo construyó una API REST en un lenguaje distinto a C#, aplicando **Onion Architecture**, **Repository Pattern** y **Unit of Work**, con las entidades Compañía y Empleado, sus migraciones, datos iniciales, los servicios, los endpoints básicos, el logging y un caso transaccional. Esa base ya funciona y respeta el flujo Controller → Servicio → Unit of Work → Repositorios → ORM → Base de datos.

La **Parte II** da continuidad a ese mismo proyecto. No se inicia un proyecto nuevo: se **amplía y refactoriza** la solución existente para acercarla a una API empresarial. Se trabajará en seis módulos consecutivos:

1. **CRUD completo:** operaciones sobre objetos individuales y sobre colecciones (creación masiva, actualización parcial, eliminación múltiple, paginación, filtrado y ordenamiento).
2. **Programación asíncrona:** investigar, implementar y refactorizar el acceso a datos y los endpoints hacia un modelo asíncrono donde la tecnología lo permita.
3. **Validaciones:** incorporar un framework o librería de validación para los datos de entrada y las reglas de negocio.
4. **Pruebas:** escribir pruebas unitarias y de integración, incluyendo la verificación del rollback transaccional.
5. **Seguridad — JWT por roles:** autenticación con JSON Web Tokens y autorización basada en roles.
6. **Seguridad — JWT por políticas:** autorización avanzada basada en políticas (policies) y claims.

Recordatorio clave

Cada grupo de 2 aprendices mantiene el lenguaje y framework asignado en la Parte I. Todo el trabajo de la Parte II se realiza sobre esa misma tecnología y sobre el mismo proyecto, conservando las capas de Onion Architecture.

1. Objetivo general

Evolucionar la API REST construida en la Parte I hacia una solución más completa y profesional, incorporando CRUD sobre colecciones, programación asíncrona, validación de datos, pruebas automatizadas y seguridad con JWT (roles y políticas), **manteniendo en todo momento la separación por capas de Onion Architecture** y el flujo a través del Unit of Work.

Meta de aprendizaje

Demostrar que conceptos avanzados de ASP.NET Core Web API —operaciones sobre colecciones, async/await, validación, pruebas y autorización por roles y políticas— son transferibles a cualquier ecosistema tecnológico, comprendiendo el principio detrás de cada uno y no solo su sintaxis.

2. Objetivos específicos

Al finalizar la actividad, el aprendiz deberá ser capaz de:

- Implementar operaciones CRUD sobre objetos individuales y sobre colecciones de entidades.
- Agregar paginación, filtrado y ordenamiento a los endpoints de listado.
- Investigar si su tecnología soporta programación asíncrona y refactorizar el código hacia ese modelo cuando corresponda.
- Aplicar un framework o librería de validación a los datos de entrada y a las reglas de negocio.
- Escribir pruebas unitarias y de integración, incluyendo la prueba del caso transaccional.
- Implementar autenticación con JWT y autorización basada en roles.
- Implementar autorización basada en políticas (policies) y claims.
- Comparar cada concepto con su equivalente en ASP.NET Core y justificar las decisiones técnicas.
- Actualizar la documentación (README) y sustentar el proyecto.

3. Recordatorio — Tecnologías asignadas

Cada grupo continúa con la combinación asignada. La tabla resume las tecnologías y el mecanismo que será relevante en esta Parte II:

Lenguaje / Framework	ORM	Mecanismo asíncrono típico	Librería de validación típica
Python con FastAPI	SQLAlchemy	async/await nativo (ASGI)	Pydantic
Python con Django	Django ORM	Vistas async / métodos async del ORM	Serializers DRF / Forms
Java con Spring Boot	JPA / Hibernate	@Async, CompletableFuture o WebFlux	Jakarta Bean Validation
Node.js con NestJS	TypeORM o Prisma	Promesas / async-await nativo	class-validator
PHP con Laravel	Eloquent	Colas/Jobs, Octane (Swoole)	Form Requests / Validator
Ruby on Rails	Active Record	Active Job / load_async	Active Record Validations
Kotlin con Spring Boot	JPA / Hibernate	Corutinas (suspend) o WebFlux	Jakarta Bean Validation
Go	GORM	goroutines + channels	go-playground/validator

Importante

Lenguaje / Framework	ORM	Mecanismo asíncrono típico	Librería de validación típica
----------------------	-----	----------------------------	-------------------------------

No todas las tecnologías ofrecen asincronía «real» de la misma forma. Parte del trabajo del Módulo 2 es justamente investigar y justificar qué modelo aplica a su stack y por qué.

4. Prompts obligatorios para usar IA (Fase II)

Como en la Parte I, los aprendices deben guardar evidencia de sus consultas con IA. Reemplacen los campos entre corchetes [] por su tecnología y ORM asignados. Estos prompts son un punto de partida; se espera que el grupo profundice con preguntas propias.

Prompt 7 — CRUD de colecciones

Prompt 7

Tengo una API REST en [lenguaje/framework] con [ORM] que ya hace CRUD de entidades individuales con Onion Architecture y Unit of Work. Explícame cómo agregar operaciones sobre colecciones: creación masiva (bulk insert), actualización parcial con PATCH, eliminación múltiple, y cómo añadir paginación, filtrado y ordenamiento al endpoint de listado, respetando las capas y sin que el repositorio confirme la transacción.

Prompt 8 — Programación asíncrona

Prompt 8

En [lenguaje/framework], ¿el manejo de peticiones y el acceso a datos con [ORM] es sincrónico o asíncrono? Si soporta asincronía, muéstrame cómo refactorizar un servicio y un repositorio de sincrónico a asíncrono (async/await, corutinas, goroutines o el mecanismo correspondiente) y qué precauciones tomar con la sesión y la transacción del Unit of Work. Si NO hay async nativo, explícame la alternativa idiomática (colas, jobs).

Prompt 9 — Validaciones

Prompt 9

¿Cuál es la librería o mecanismo de validación recomendado para [lenguaje/framework]? Muéstrame cómo validar los DTOs de Compañía y Empleado (campos obligatorios, longitud, formato de correo, salario positivo) y cómo devolver el error de validación con el código HTTP adecuado, manteniendo la validación en la capa de aplicación.

Prompt 10 — Pruebas

Prompt 10

¿Cuál es el framework de pruebas más usado en [lenguaje/framework]?
Muéstrame cómo escribir pruebas unitarias para un servicio y pruebas de integración para los endpoints de Compañía y Empleado, incluyendo una prueba que verifique el rollback del endpoint transaccional cuando falla la creación de un empleado.

Prompt 11 — JWT por roles

Prompt 11

Explicame cómo implementar autenticación con JWT en [lenguaje/framework]: entidad Usuario, registro, login que devuelve un token y cómo proteger endpoints exigiendo un rol específico (p. ej. solo ADMIN puede eliminar). Indica la librería recomendada y dónde ubicar cada parte en Onion Architecture.

Prompt 12 — JWT por políticas

Prompt 12

¿Cuál es la diferencia entre autorización por roles y por políticas (claims/policies) en [lenguaje/framework]? Muéstrame cómo implementar una política basada en reglas o claims (ej.: un usuario solo puede editar empleados de su propia compañía) y compárala con las policies de ASP.NET Core ([Authorize(Policy=...)] con requirements y handlers).

5. Módulo 1 — CRUD completo: objetos y colecciones

En la Parte I se construyó el CRUD sobre **objetos individuales**: crear un empleado, consultar uno por id, actualizarlo o eliminarlo. En este módulo se completa el CRUD para que la API también opere sobre **colecciones**, es decir, sobre varios recursos a la vez, y para que los listados sean usables con muchos registros.

5.1 Objeto vs. colección

La diferencia es conceptual y se refleja en la ruta, el cuerpo de la petición y el código de respuesta:

- **Recurso individual:** la ruta apunta a un único elemento (/api/empleados/{id}). El cuerpo describe un solo objeto.
- **Colección:** la ruta apunta al conjunto (/api/empleados). El cuerpo o la respuesta contienen una lista, y aparecen parámetros de consulta para paginar, filtrar y ordenar.

5.2 Operaciones sobre colecciones

El módulo exige implementar, además del CRUD individual ya existente, las siguientes operaciones de colección:

- **Creación masiva (bulk):** recibir una lista de empleados y crearlos todos dentro de una misma transacción del Unit of Work.
- **Actualización parcial (PATCH):** modificar solo algunos campos de un recurso, en lugar de reemplazarlo por completo como hace PUT.
- **Eliminación múltiple:** eliminar varios recursos identificados por una lista de ids.
- **Listado avanzado:** paginación, filtrado y ordenamiento sobre el endpoint de listado.

5.3 Paginación, filtrado y ordenamiento

Un listado sin límites es un problema en producción: devolver miles de registros es lento y costoso. Por eso el endpoint de listado debe aceptar parámetros de consulta (*query parameters*). Ejemplo de ruta:

```
GET /api/empleados?pagina=1&tamano=10&orden=apellido&dir=asc&buscar=gomez
```

pagina → número de página (empezando en 1)
 tamano → cantidad de registros por página
 orden → campo por el cual ordenar (apellido, salario, ...)
 dir → dirección del orden (asc | desc)
 buscar → texto para filtrar por nombre, apellido o correo

La respuesta de un listado paginado debe envolver los datos junto con la información de paginación (sobre, o «envelope»):

```
{
  "datos": [ { ...empleado... }, { ...empleado... } ],
  "pagina": 1,
  "tamano": 10,
  "total": 57,
  "totalPaginas": 6
}
```

5.4 Endpoints adicionales requeridos

A los endpoints de la Parte I se suman, como mínimo, los siguientes:

Método	Ruta	Descripción
GET	/api/empleados?pagina=&tamano=&orden=&dir=&buscar=	Listado paginado, filtrado y ordenado
GET	/api/companias/{id}/empleados?pagina=&tamano=	Empleados de una compañía, paginados
POST	/api/empleados/lote	Creación masiva de empleados (bulk)
PATCH	/api/empleados/{id}	Actualización parcial de un empleado
DELETE	/api/empleados/lote	Eliminación múltiple (lista de ids en el body)

Regla de arquitectura

Método	Ruta	Descripción
La creación masiva y la eliminación múltiple deben ejecutarse dentro de UNA sola transacción del Unit of Work: o se aplican todos los cambios, o no se aplica ninguno. El repositorio agrega los cambios; el Unit of Work confirma (commit) o revierte (rollback).		

5.5 Códigos HTTP para colecciones

Código	Uso en colecciones
200 OK	Listado devuelto / actualización parcial exitosa
201 Created	Uno o varios recursos creados (bulk)
204 No Content	Eliminación múltiple exitosa sin cuerpo de respuesta
207 Multi-Status	(Opcional) Resultado mixto en operaciones por lotes
400 Bad Request	Parámetros de paginación/filtro inválidos
404 Not Found	Alguno de los recursos solicitados no existe

5.6 Pasos del módulo

Paso 15 · Ampliar los repositorios para operaciones de colección

Agregar a los repositorios métodos que operen sobre listas, complementando los de la Parte I (GetAll, GetById, Create, Update, Delete, FindByCondition):

- **CreateRange(lista)** — agrega varios registros (sin confirmar).
- **DeleteRange(ids)** — marca varios registros para eliminación.
- **GetPaged(pagina, tamaño, orden, dir, filtro)** — devuelve una página de resultados y el total.
- **PatchPartial(id, cambios)** — aplica solo los campos presentes.

Recuerde

Los repositorios NO deben llamar a save, commit ni flush. Esa responsabilidad es exclusiva del Unit of Work.

Paso 16 · Implementar paginación, filtrado y ordenamiento

En el repositorio (capa Infrastructure), construir la consulta del ORM aplicando, en este orden: filtro de búsqueda, ordenamiento y, por último, el desplazamiento de página. Documentar cómo su ORM expresa cada parte:

- Filtro: cláusula de tipo WHERE / where / filter sobre nombre, apellido o correo.
- Ordenamiento: ORDER BY dinámico según el campo y la dirección recibidos.
- Paginación: desplazamiento (offset/skip) y límite (limit/take); calcular el total de registros.

Paso 17 · Crear los endpoints de colección

En la capa API/Presentation, exponer los endpoints de la tabla 5.4. El controlador recibe los parámetros de consulta y el cuerpo, los pasa al servicio y devuelve el código HTTP adecuado. El servicio coordina el Unit of Work para las operaciones por lotes. Probar cada endpoint con Swagger, Postman, Thunder Client o Insomnia y guardar capturas.

6. Módulo 2 — Programación asíncrona

La programación asíncrona permite que la aplicación atienda otras peticiones mientras espera operaciones lentas (consultas a base de datos, llamadas de red), en lugar de bloquear el hilo. En APIs con muchas peticiones concurrentes, esto mejora notablemente el rendimiento y el uso de recursos.

6.1 ¿Por qué importa?

- **Concurrencia:** más peticiones atendidas con los mismos recursos.
- **Escalabilidad:** mejor aprovechamiento del hilo mientras se espera I/O.
- **Equivalencia con ASP.NET Core:** en C# esto se expresa con `async/await` y `Task<T>`; el objetivo es encontrar el equivalente idiomático en su tecnología.

6.2 Investigación por tecnología

Antes de refactorizar, el grupo debe investigar y documentar el modelo asíncrono de su stack:

Tecnología	¿Async nativo?	Mecanismo principal	Nota sobre el ORM
FastAPI	Sí	<code>async def</code> + <code>await</code> (ASGI)	SQLAlchemy <code>async</code> (<code>AsyncSession</code>) o ejecución en <code>threadpool</code>
Django	Parcial	Vistas <code>async</code> ; métodos <code>async</code> del ORM (<code>acreate</code> , <code>aget</code> ...)	Muchas operaciones siguen siendo sincrónicas
Spring Boot (Java)	No en MVC	@Async + <code>CompletableFuture</code> , o <code>WebFlux</code> (reactivo)	JPA es bloqueante; reactivo requiere R2DBC
NestJS	Sí	Promesas / <code>async-await</code>	TypeORM y Prisma ya devuelven Promesas
Laravel	No nativo	Colas/Jobs, Octane (Swoole/RoadRunner)	Eloquent es sincrónico; concurrencia vía colas
Rails	Limitado	Active Job (Sidekiq), <code>load_async</code>	Active Record mayormente sincrónico
Kotlin Spring	Sí (corutinas)	<code>suspend fun</code> + corutinas, o <code>WebFlux</code>	JPA bloqueante; reactivo con R2DBC
Go	Sí	<code>goroutines</code> + <code>channels</code>	GORM bloquea por llamada, pero corre dentro de <code>goroutines</code>

Tecnología	¿Async nativo?	Mecanismo principal	Nota sobre el ORM
------------	----------------	---------------------	-------------------

Honestidad técnica

Si su tecnología NO ofrece asincronía real para el acceso a datos (por ejemplo, Laravel o Rails con su ORM clásico), NO inventen un async ficticio. Documenten esa limitación y expliquen la alternativa idiomática: colas, jobs en segundo plano o ejecución concurrente a nivel de servidor. Esa explicación correcta vale tanto como una refactorización.

6.3 Refactorización: de sincrónico a asíncrono

La refactorización recorre las capas de afuera hacia adentro, manteniendo Onion Architecture intacta:

```

Controller / Route      (handler asíncrono)
  ↓ await
Service                (método asíncrono)
  ↓ await
Unit of Work           (commit / rollback asíncrono)
  ↓ await
Repositories           (consultas asíncronas)
  ↓ await
ORM Context / Session  (sesión asíncrona)
  ↓
Database

```

6.4 Async y Unit of Work

Al pasar a asíncrono, la sesión o contexto del Unit of Work debe ser compatible con ese modelo. Puntos de cuidado:

- Usar la sesión/contexto asíncrono correspondiente (p. ej. AsyncSession en SQLAlchemy).
- Una sesión asíncrona normalmente NO debe compartirse entre tareas concurrentes: una unidad de trabajo, una sesión.
- El commit y el rollback también se vuelven asíncronos (se espera con await).
- El caso transaccional de la Parte I debe seguir funcionando: si falla un empleado, no se guarda nada.

6.5 Pasos del módulo

Paso 18 · Investigar el soporte asíncrono de su tecnología

Responder y documentar: ¿el framework atiende peticiones de forma asíncrona? ¿El ORM permite consultas asíncronas? ¿Qué objeto representa la sesión/contexto asíncrono? Si no hay async nativo, ¿cuál es la alternativa?

Paso 19 · Refactorizar repositorios y servicios

Convertir los métodos de los repositorios y servicios a su forma asíncrona (async/await, suspend, goroutines o el mecanismo aplicable). Si su stack no soporta async en datos, implementar la alternativa (por ejemplo, mover la creación masiva a un job) y explicarlo.

Paso 20 · Ajustar el Unit of Work y las transacciones

Adaptar el Unit of Work al modelo asíncrono y volver a probar el endpoint transaccional para confirmar que el commit y el rollback siguen siendo correctos bajo asincronía. Guardar evidencia de la prueba.

7. Módulo 3 — Validaciones

Hasta ahora la API confía en que los datos que llegan son correctos. En este módulo se incorpora la **validación**: rechazar de forma controlada los datos inválidos antes de que lleguen a la base de datos, devolviendo un mensaje claro y el código HTTP adecuado.

7.1 ¿Por qué validar en la capa de aplicación?

La validación pertenece a la capa Application/Services (sobre los DTOs), no al controlador ni al ORM. Así se garantiza que cualquier punto de entrada respete las mismas reglas y que el dominio reciba siempre datos consistentes. Se distinguen dos niveles:

- **Validación de formato/estructura:** campos obligatorios, longitud, tipo, formato de correo, salario positivo.
- **Validación de reglas de negocio:** p. ej., el correo del empleado no debe estar repetido; la compañía referenciada debe existir.

7.2 Librería de validación por tecnología

Tecnología	Librería / mecanismo	Cómo se expresa
FastAPI	Pydantic	Modelos con tipos + Field(...) y validadores (field_validator)
Django	Serializers (DRF) / Forms	Campos con required, validadores y validate_<campo>()
Spring Boot (Java)	Jakarta Bean Validation (Hibernate Validator)	Anotaciones @NotNull, @Email, @Size, @Positive + @Valid
NestJS	class-validator + class-transformer	Decoradores @IsString, @IsEmail... + ValidationPipe
Laravel	Form Requests / Validator	Reglas 'required email max:255' en un FormRequest
Rails	Active Record / Active Model	validates :campo, presence: true, format: { ... }
Kotlin Spring	Jakarta Bean Validation	Anotaciones sobre data classes + @Valid

Tecnología	Librería / mecanismo	Cómo se expresa
Go	go-playground/validator	Tags de struct: `validate:"required,email"``

7.3 Validaciones mínimas requeridas

Aplicar sobre los DTOs de Compañía y Empleado al menos las siguientes reglas:

Entidad	Campo	Regla
Compañía	Nombre	Obligatorio, entre 3 y 100 caracteres
Compañía	Telefono	Obligatorio, solo dígitos, longitud válida
Empleado	Nombre / Apellido	Obligatorios, no vacíos
Empleado	Correo	Obligatorio, formato de correo válido, único
Empleado	Salario	Obligatorio, numérico y mayor que 0
Empleado	CompaniaId	Obligatorio; la compañía debe existir

7.4 Manejo de errores de validación

Cuando la validación falla, la API no debe romperse: debe responder de forma uniforme con el código adecuado (usualmente **400 Bad Request** o **422 Unprocessable Entity**) y un cuerpo que detalle qué campos fallaron:

```
HTTP 422 Unprocessable Entity
{
  "mensaje": "Error de validación",
  "errores": [
    { "campo": "correo", "detalle": "Formato de correo inválido" },
    { "campo": "salario", "detalle": "Debe ser mayor que 0" }
  ]
}
```

7.5 Pasos del módulo

Paso 21 · Seleccionar e instalar la librería de validación

Identificar la librería de la tabla 7.2 para su stack, instalarla y documentar el comando usado. Si es nativa del framework, indicarlo.

Paso 22 · Validar los DTOs y las reglas de negocio

Aplicar las validaciones de la tabla 7.3 a los DTOs de la capa Application. Implementar al menos una regla de negocio (correo único o existencia de la compañía) que requiera consultar el repositorio.

Paso 23 · Manejar y formatear los errores

Centralizar el manejo de errores de validación (middleware, filtro, exception handler o equivalente) para devolver siempre el mismo formato y código HTTP. Probar con datos inválidos y guardar capturas.

8. Módulo 4 — Pruebas

Las pruebas automatizadas verifican que el código hace lo que debe y protegen contra regresiones al refactorizar. En este módulo se cubren al menos dos niveles: pruebas **unitarias** (lógica aislada) e pruebas de **integración** (endpoints contra una base de datos de prueba).

8.1 Pirámide de pruebas

- **Unitarias (base):** muchas, rápidas; prueban un servicio o un repositorio de forma aislada (con dobles/mocks si hace falta).
- **Integración (medio):** prueban varias capas juntas, normalmente los endpoints contra una base de datos de prueba (en memoria o efímera).
- **Extremo a extremo / E2E (cima):** pocas; recorren el sistema completo. Opcionales en esta actividad.

8.2 Framework de pruebas por tecnología

Tecnología	Framework de pruebas	Apoyo (integración / mocks)
FastAPI	pytest	TestClient / httpx, fixtures, BD de prueba
Django	pytest-django / unittest	APIClient (DRF), BD de prueba
Spring Boot (Java)	JUnit 5	Mockito, MockMvc, @SpringBootTest, Testcontainers
NestJS	Jest	Supertest (e2e), @nestjs/testing
Laravel	PHPUnit / Pest	Factories, RefreshDatabase, TestCase
Rails	RSpec / Minitest	FactoryBot, request specs
Kotlin Spring	JUnit 5 + MockK	Kotest opcional, MockMvc / WebTestClient
Go	testing (stdlib)	testify, httptest, sqlmock

8.3 Qué se debe probar

- Servicios: que CompaniaService y EmpleadoService devuelvan y transformen datos correctamente.
- Repositorios: operaciones CRUD y la consulta paginada/filtrada del Módulo 1.
- Endpoints: que respondan los códigos HTTP correctos (200, 201, 204, 400/422, 404).
- Validaciones: que los datos inválidos sean rechazados con el formato esperado.
- Caso transaccional: que el rollback funcione cuando falla la creación de un empleado.

8.4 Prueba del caso transaccional (rollback)

Prueba obligatoria

Crear una compañía con varios empleados donde uno sea inválido (por ejemplo, correo duplicado o salario negativo). La prueba debe verificar que, tras el fallo, NO quedó guardada ni la compañía ni ningún empleado. Esta es la evidencia central de que el Unit of Work controla la transacción.

8.5 Pasos del módulo

Paso 24 · Configurar el entorno de pruebas

Instalar el framework de la tabla 8.2 y configurar una base de datos de prueba (en memoria, SQLite de prueba o contenedor efímero). Documentar el comando para ejecutar las pruebas.

Paso 25 · Escribir pruebas unitarias

Probar al menos un método de un servicio y uno de un repositorio. Usar doubles/mocks cuando sea necesario para aislar la lógica.

Paso 26 · Escribir pruebas de integración

Probar los endpoints principales (listado paginado, creación, actualización parcial, eliminación) verificando el código HTTP y el cuerpo de la respuesta.

Paso 27 · Probar el caso transaccional

Implementar la prueba descrita en 8.4 y confirmar el rollback. Guardar la salida de la ejecución de pruebas como evidencia.

9. Módulo 5 — Seguridad: JWT orientado a Roles

Hasta ahora cualquiera puede consumir la API. En este módulo se incorpora seguridad con **JSON Web Tokens (JWT)**: el usuario se autentica, recibe un token firmado y lo envía en cada petición; la API verifica el token y autoriza según el **rol** del usuario.

9.1 Autenticación vs. autorización

- **Autenticación:** verificar quién es el usuario (login).
- **Autorización:** decidir qué puede hacer ese usuario (roles, permisos).

9.2 ¿Qué es un JWT?

Un JWT es una cadena firmada con tres partes separadas por puntos: **encabezado.payload.firma**. El *payload* contiene «claims» (afirmaciones) como el id del usuario, su rol y la expiración. La firma garantiza que el token no fue alterado. El servidor no guarda el token: lo verifica con su clave secreta en cada petición.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9. <payload>.<firma>
```



```

payload (ejemplo):
{
  "sub": 12,           // id del usuario
  "correo": "ana@x.com",
  "rol": "ADMIN",      // claim de rol
  "exp": 1735689600    // expiración
}

```

9.3 Nuevas entidades: Usuario y Rol

Se agrega al dominio la entidad Usuario. El rol puede ser un campo (enum) o una entidad relacionada:

Campo	Descripción
Id	Llave primaria
Nombre	Nombre del usuario
Correo	Correo electrónico (único, usado para login)
ContrasenaHash	Hash de la contraseña (NUNCA texto plano)
Rol	ADMIN o USUARIO (o entidad Rol relacionada)
CompaniaId	(Opcional) compañía a la que pertenece — se usará en el Módulo 6
FechaCreacion	Fecha de registro

Seguridad de contraseñas

La contraseña jamás se almacena ni se transmite en texto plano. Se guarda solo su hash, usando un algoritmo adecuado (bcrypt, argon2, PBKDF2 o el recomendado por su stack). El login compara el hash, no la contraseña original.

9.4 Flujo de autenticación

1. POST /api/auth/registro → crea usuario (guarda hash)
2. POST /api/auth/login → valida credenciales y devuelve un JWT
3. El cliente envía el token en el header Authorization: Bearer <token>
4. Un middleware / guard / filtro valida el token en cada petición
5. Si el token es válido, se autoriza según el rol; si no, 401 / 403

9.5 Endpoints de autenticación

Método	Ruta	Descripción	Protección
POST	/api/auth/registro	Registrar un usuario	Pública
POST	/api/auth/login	Iniciar sesión y obtener JWT	Pública
GET	/api/auth/perfil	Datos del usuario autenticado	Requiere token

Y se protegen los endpoints existentes según el rol. Propuesta mínima:

Operación	Rol requerido
GET (listar / consultar)	Cualquier usuario autenticado
POST (crear) y PUT/PATCH (actualizar)	ADMIN o USUARIO

Operación	Rol requerido
DELETE (eliminar)	Solo ADMIN
POST /api/companias/con-empleados (transaccional)	Solo ADMIN

9.6 Roles por tecnología

Tecnología	Librería JWT	Mecanismo de roles
FastAPI	python-jose / PyJWT + passlib	OAuth2PasswordBearer + dependencia que valida el rol
Django	djangorestframework-simplejwt	Permission classes + grupos/roles
Spring Boot (Java)	Spring Security + jjwt/nimbus	@PreAuthorize("hasRole('ADMIN')")
NestJS	@nestjs/jwt + passport-jwt	JwtAuthGuard + RolesGuard + @Roles()
Laravel	Sanctum / Passport / jwt-auth	Middleware + spatie/laravel-permission
Rails	jwt (gem)	before_action + verificación del rol en claims
Kotlin Spring	Spring Security + JWT	@PreAuthorize con hasRole
Go	golang-jwt/jwt	Middleware que valida token y rol del claim

9.7 Ubicación en Onion Architecture

Domain	→ entidad Usuario, interfaz IUserRepository
Application	→ AuthService (registro, login), DTOs, interfaz del servicio de tokens y de hashing (abstracciones)
Infraestructure	→ implementación del repositorio, generación/validación del JWT, hashing de contraseñas
API/Presentation	→ AuthController, middleware/guard/filtro de seguridad

9.8 Pasos del módulo

Paso 28 · Crear la entidad Usuario y el rol

Agregar la entidad Usuario al dominio, su migración y un usuario ADMIN inicial en los datos semilla.

Paso 29 · Implementar registro y hash de contraseña

Crear el endpoint de registro que guarde el hash de la contraseña. Documentar el algoritmo de hashing usado.

Paso 30 · Implementar login y emisión del JWT

Validar las credenciales y emitir un token firmado con los claims de id, correo, rol y expiración. Documentar la clave secreta como variable de entorno (NUNCA en el código).

Paso 31 · Validar el token en cada petición

Implementar el middleware/guard/filtro que verifique la firma y la expiración del token, y que ponga el usuario autenticado a disposición de los controladores.

Paso 32 · Restringir endpoints por rol

Aplicar la matriz de roles de 9.5 a los endpoints. Probar accesos permitidos (200) y denegados (401 sin token, 403 con rol insuficiente). Guardar capturas.

10. Módulo 6 — Seguridad: JWT orientado a Políticas (Policies)

Los roles responden «qué eres» (ADMIN, USUARIO). Las **políticas** responden «qué se te permite hacer en esta situación concreta», evaluando reglas y claims. Es el mismo salto que en ASP.NET Core se da de `[Authorize(Roles="...")]` a `[Authorize(Policy="...")]` con requirements y handlers.

10.1 Roles vs. políticas

Aspecto	Por roles	Por políticas
Pregunta	¿Qué rol tiene el usuario?	¿Cumple las condiciones para esta acción?
Basado en	Un rol fijo	Claims, atributos del recurso y reglas
Granularidad	Gruesa	Fina (incluso a nivel de cada objeto)
Ejemplo	Solo ADMIN elimina	Solo el dueño de la compañía edita SUS empleados

10.2 Autorización basada en claims y reglas

Una política toma una decisión (permitir/denegar) combinando: los claims del token (id, rol, compañía), datos del recurso solicitado y reglas del negocio. La política se evalúa en el momento de la petición, no se fija de antemano.

10.3 Políticas a implementar

Implementar al menos UNA política de propiedad (ownership) y, opcionalmente, una basada en un claim de permiso:

- **EsPropietarioDeCompania** — un USUARIO solo puede actualizar o eliminar empleados cuya `CompaniaId` coincide con la compañía de su token (el ADMIN no tiene esta restricción).
- **LimiteSalario** — solo un ADMIN puede asignar salarios por encima de un umbral definido.
- **Permiso por claim** — el token incluye un claim como permisos: `["empleados:eliminar"]` y la política exige ese permiso.

10.4 Políticas por tecnología

Tecnología	Mecanismo de políticas
ASP.NET Core (referencia)	AddAuthorization + AuthorizationHandler/Requirement; [Authorize(Policy=...)]
FastAPI	Dependencias compuestas que evalúan claims y recurso; o Casbin / oso
Django	has_object_permission, django-guardian (objeto), o oso
Spring Boot (Java)	@PreAuthorize con SpEL, PermissionEvaluator personalizado
NestJS	CASL (ability) + PoliciesGuard con @CheckPolicies
Laravel	Gates y Policies (make:policy), método can/authorize, before()
Rails	Pundit (policies) o CanCanCan (abilities)
Kotlin Spring	@PreAuthorize con SpEL (igual que Java)
Go	Casbin (RBAC/ABAC) en middleware de autorización

10.5 Comparación con ASP.NET Core

En ASP.NET Core una policy se registra y luego se aplica de forma declarativa. El objetivo es identificar la pieza equivalente en su stack:

```
// ASP.NET Core (referencia conceptual)
builder.Services.AddAuthorization(o =>
    o.AddPolicy("EsPropietario",
        p => p.Requirements.Add(new EsPropietarioRequirement())));

[Authorize(Policy = "EsPropietario")]
public IActionResult ActualizarEmpleado(...) { ... }

// EsPropietarioHandler evalúa el claim de compañía contra el recurso.
```

10.6 Pasos del módulo

Paso 33 · Definir las políticas

Describir, en lenguaje natural y luego como regla, cada política a implementar (al menos la de propiedad). Identificar qué claims debe incluir el token para evaluarlas.

Paso 34 · Implementar la evaluación de políticas

Crear los handlers/gates/policies/abilities según su stack (tabla 10.4) que evalúen claims, recurso y reglas, devolviendo permitir o denegar.

Paso 35 · Aplicar las políticas a los endpoints

Proteger los endpoints de actualización/eliminación de empleados con la política de propiedad, dejando al ADMIN exento. Mantener los roles del Módulo 5 como primera barrera.

Paso 36 · Probar y documentar las políticas

Probar: el dueño puede editar sus empleados (200); otro usuario no puede (403); el ADMIN puede todo. Guardar capturas y explicar la implementación en el README.

11. Producto final esperado (Parte II)

Sobre el proyecto de la Parte I, cada grupo debe entregar adicionalmente:

1. Código fuente actualizado con los seis módulos integrados.
2. Endpoints de colección funcionando (bulk, PATCH, eliminación múltiple, listado paginado).
3. Evidencia de la refactorización asíncrona (o de la alternativa, si no hay async nativo) con justificación.
4. Validaciones aplicadas y captura de una respuesta de error de validación.
5. Suite de pruebas ejecutándose, con la prueba del rollback transaccional.
6. Autenticación JWT funcionando: registro, login y endpoints protegidos por rol.
7. Al menos una política (policy) implementada y aplicada a un endpoint.
8. README actualizado (ver sección 12).
9. Capturas de pruebas de cada módulo (Swagger/Postman/Thunder Client/Insomnia).
10. Evidencia del uso de IA (prompts 7 a 12 y consultas propias).
11. Tabla comparativa ampliada con ASP.NET Core (sección 14).
12. Exposición corta del proyecto.

12. Actualización del README

Al README.md de la Parte I se le agregan las siguientes secciones:

```
## CRUD de colecciones
### Paginación, filtrado y ordenamiento
## Programación asíncrona
### ¿Mi tecnología soporta async? (sí/no y por qué)
### Qué se refactorizó (o alternativa aplicada)
## Validaciones
### Librería usada y reglas aplicadas
## Pruebas
### Cómo ejecutar las pruebas
### Prueba del rollback transaccional
## Seguridad
### Autenticación con JWT
### Autorización por roles
### Autorización por políticas (policies)
## Variables de entorno (clave secreta JWT, conexión, etc.)
## Comparación ampliada con ASP.NET Core
## Conclusiones de la Parte II
```

13. Criterios de evaluación (Parte II)

Criterio	Puntaje
CRUD de objetos y colecciones (bulk, PATCH, eliminación múltiple)	15

Criterio	Puntaje
Paginación, filtrado y ordenamiento	10
Programación asíncrona o alternativa justificada	15
Validaciones y manejo de errores	10
Pruebas unitarias y de integración	10
Prueba del caso transaccional (rollback)	5
JWT y autorización por roles	15
Autorización por políticas (policies)	10
Documentación, comparación con ASP.NET Core y uso de IA	5
Sustentación	5
TOTAL	100

14. Comparación ampliada con ASP.NET Core

Completar la columna derecha con el equivalente en la tecnología asignada. Esta tabla extiende la de la Parte I con los conceptos nuevos:

Concepto en ASP.NET Core	Equivalente en la tecnología asignada
Endpoints de colección (IEnumerable / List)	_____
Paginación (Skip/Take)	_____
async / await + Task<T>	_____
DataAnnotations / FluentValidation	_____
xUnit / NUnit + Moq	_____
AddAuthentication().AddJwtBearer()	_____
[Authorize(Roles = "...")]	_____
[Authorize(Policy = "...")] + IAuthorizationHandler	_____
ClaimsPrincipal / Claims	_____

15. Preguntas para sustentación (Parte II)

Cada aprendiz debe estar preparado para responder, además de las de la Parte I:

1. ¿Qué diferencia hay entre operar sobre un objeto y sobre una colección?
2. ¿Cómo garantiza que una creación masiva se guarde como una sola transacción?
3. ¿Cómo implementó la paginación y por qué es importante en producción?
4. ¿Su tecnología soporta asincronía real? ¿Qué refactorizó o qué alternativa aplicó?
5. ¿Qué precauciones tomó con la sesión del Unit of Work al pasar a asíncrono?
6. ¿Qué librería de validación usó y en qué capa ubicó las validaciones?

7. ¿Qué probó con pruebas unitarias y qué con pruebas de integración?
8. ¿Cómo demostró el rollback del caso transaccional en una prueba?
9. ¿Qué es un JWT y qué contiene su payload?
10. ¿Por qué nunca se almacena la contraseña en texto plano?
11. ¿Cuál es la diferencia entre autorización por roles y por políticas?
12. ¿Cómo implementó la política de propiedad (ownership) y cómo la comparó con las políticas de ASP.NET Core?

16. Reglas técnicas obligatorias (Parte II)

La solución debe seguir demostrando el flujo por capas, ahora con los nuevos elementos:

```
Controller / Route → Service → Unit of Work → Repositories → ORM → DB
      ↑
  Middleware/Guard de seguridad (JWT) y validación ANTES del Service
```

No se acepta una solución donde:

- El controlador acceda directamente al ORM o a los repositorios concretos.
- Cada repositorio confirme su propia transacción sin pasar por el Unit of Work.
- Las validaciones vivan solo en el controlador o en la base de datos.
- La contraseña se guarde o se transmita en texto plano.
- La clave secreta del JWT esté escrita directamente en el código fuente.
- Los endpoints sensibles (eliminar, transaccional) queden sin protección de rol o política.

Resultado esperado de aprendizaje

Al finalizar la Parte II

El aprendiz será capaz de evolucionar una API REST hacia una solución cercana a la producción —con CRUD de colecciones, asincronía donde aplique, validación, pruebas automatizadas y seguridad con JWT por roles y por políticas— comprendiendo que estos principios, aprendidos en ASP.NET Core Web API con C#, son transferibles a cualquier ecosistema tecnológico y se sostienen sobre la misma arquitectura: Onion Architecture, Repository Pattern y Unit of Work.